Smart Contract
Security Audit Report

Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

4 Code Overview

4.1 Contracts Description

4.2 Visibility Description

4.3 Vulnerability Summary

5 Audit Result

6 Statement

1 Executive Summary

On 2026.02.27, the SlowMist security team received the Sigma Money team's security audit application for Sigma Dao round 6 & Money round 8, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

Sigma DAO protocol is forked from Shadow Protocol; Sigma Money protocol is forked from Fx Protocol and Pendle finance.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Risks of excessive privilege	Authority Control Vulnerability Audit	Medium	Acknowledged

NO	Title	Category	Level	Status
N2	Redundant code	Others	Suggestion	Fixed
N3	Owner transfer not synchronized AccessControl role	Authority Control Vulnerability Audit	High	Fixed
N4	Missing zero address check	Others	Suggestion	Fixed
N5	Missing event log	Others	Suggestion	Fixed
N6	Ignore return value	Others	Suggestion	Fixed
N7	Missing array length consistency check	Others	Suggestion	Fixed
N8	The risk of conflict when both initializeV2() and initialize() use the initializer modifier	Design Logic Audit	High	Fixed

4 Code Overview

4.1 Contracts Description

Part 1:

https://github.com/SigmaMoney/dao/tree/feat/gaugeV2

Initial audit version: 24a662e3571e9cc04270ad9eeea2683bc5e173ca

Final audit version: 2aa90917206aafa1c8140ead84c646ce4e3ac358

Audit scope:

```
contracts
├── airdrop
│   └── SigmaAirdrop.sol
├── factories
│   ├── SigmaFeeDistributorFactory.sol
│   ├── SigmaGaugeFactory.sol
│   └── VeFunderGaugeFactory.sol
├── interfaces
│   ├── IAccessHub.sol
```

```
│   ├── IERC20Extended.sol
│   ├── IFeeDistributor.sol
│   ├── IGauge.sol
│   ├── IGaugeEmission.sol
│   ├── ISigmaFeeDistributor.sol
│   ├── ISigmaFeeDistributorFactory.sol
│   ├── ISigmaGauge.sol
│   ├── ISigmaGaugeFactory.sol
│   ├── IVeFunderGauge.sol
│   ├── IVeFunderGaugeFactory.sol
│   ├── IVoteModule.sol
│   ├── IVoter.sol
│   ├── IX33.sol
│   └── IXShadow.sol
├── libraries
│   └── RewardClaimers.sol
├── AccessHub.sol
├── GaugeEmission.sol
├── SigmaFeeDistributor.sol
├── SigmaGauge.sol
├── TimelockedTransparentUpgradableProxy.sol
├── Token.sol
├── VeFunderGauge.sol
├── VoteModule.sol
├── Voter.sol
└── xShadow
    ├── x33.sol
    └── XShadow.sol
```

Part 2:

https://github.com/SigmaMoney/contracts/tree/feat/bsc

Initial audit version: 41108bc2becda794a5e9db44ddcb51b50e441287

Final audit version: d57fda7077e4c5c3eb527af61cd2e3a688a25736

Audit scope:

```
- contracts/core/PoolManager.sol
- contracts/fund/strategy/StrategyBaseUpgradeable.sol
- contracts/fund/strategy/YieldStrategy.sol
- contracts/periphery/facets/ShortPositionOperateFacet.sol
```

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

AccessHub			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
reinit	External	Can Modify State	onlyOwner
initializeVoter	External	Can Modify State	onlyOwner
startRebase	External	Can Modify State	onlyRole
setNewGovernorInVoter	External	Can Modify State	onlyRole
createSigmaGauge	External	Can Modify State	onlyRole
createVeFunderGauge	External	Can Modify State	onlyRole
governanceWhitelist	External	Can Modify State	onlyRole
killGauge	External	Can Modify State	onlyRole
reviveGauge	External	Can Modify State	onlyRole
setEmissionsRatioInVoter	External	Can Modify State	onlyRole
retrieveStuckEmissionsToGovernance	External	Can Modify State	onlyRole
setGaugePreallocation	External	Can Modify State	onlyRole
transferWhitelistInXShadow	External	Can Modify State	onlyRole
toggleXShadowGovernance	External	Can Modify State	onlyRole
operatorRedeemXShadow	External	Can Modify State	onlyRole
migrateOperator	External	Can Modify State	onlyRole
rescueTrappedTokens	External	Can Modify State	onlyRole

AccessHub			
setExemptionToInXShadow	External	Can Modify State	onlyRole
kickoffGaugeEmission	External	Can Modify State	onlyRole
startGaugeEmissions	External	Can Modify State	onlyRole
setEmissionsMultiplier	External	Can Modify State	onlyRole
setGaugeActive	External	Can Modify State	onlyRole
emergencyWithdrawFromGaugeEmission	External	Can Modify State	onlyOwner
augmentGaugeRewardsForPair	External	Can Modify State	onlyRole
removeFeeDistributorRewards	External	Can Modify State	onlyRole
setCooldownExemption	External	Can Modify State	onlyOwner
setNewRebaseStreamingDuration	External	Can Modify State	onlyOwner
setNewVoteModuleCooldown	External	Can Modify State	onlyOwner
kickInactive	External	Can Modify State	onlyRole
execute	External	Can Modify State	onlyOwner
setNewOwner	External	Can Modify State	onlyOwner
setNewOperator	External	Can Modify State	onlyOwner
upgradeContract	External	Can Modify State	onlyOwner

GaugeEmission			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyGovernance
kickoff	External	Can Modify State	onlyGovernance

GaugeEmission			
setGaugeActive	External	Can Modify State	onlyGovernance
updatePeriod	External	Can Modify State	-
startEmissions	External	Can Modify State	onlyGovernance
updateEmissionsMultiplier	External	Can Modify State	onlyGovernance
calculateWeeklyEmissions	Public	-	-
getPeriod	Public	-	-
getEpoch	Public	-	-
emergencyWithdraw	External	Can Modify State	onlyGovernance
_safeTransfer	Internal	Can Modify State	-

SigmaFeeDistributor			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
_deposit	External	Can Modify State	-
_withdraw	External	Can Modify State	-
getPeriodReward	External	Can Modify State	nonReentrant
getReward	External	Can Modify State	nonReentrant
getRewardForOwner	External	Can Modify State	nonReentrant
incentivize	External	Can Modify State	nonReentrant
removeReward	External	Can Modify State	-
getRewardTokens	External	-	-
earned	External	-	-
getPeriod	Public	-	-

SigmaFeeDistributor			
_getReward	Internal	Can Modify State	-
_getAllRewards	Internal	Can Modify State	-
_safeTransfer	Internal	Can Modify State	-
_safeTransferFrom	Internal	Can Modify State	-

Gauge			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
rewardsList	External	-	-
rewardsListLength	External	-	-
lastTimeRewardApplicable	Public	-	-
rewardData	External	-	-
earned	Public	-	-
rewardPerToken	Public	-	-
left	Public	-	-
isWhitelisted	Public	-	-
getReward	Public	Can Modify State	updateReward nonReentrant
getRewardAndExit	Public	Can Modify State	updateReward nonReentrant
depositAll	External	Can Modify State	-
depositFor	Public	Can Modify State	updateReward nonReentrant
deposit	Public	Can Modify State	-
withdrawAll	External	Can Modify State	-
withdraw	Public	Can Modify State	updateReward nonReentrant

Gauge			
whitelistReward	External	Can Modify State	-
removeRewardWhitelist	External	Can Modify State	-
notifyRewardAmount	External	Can Modify State	updateReward nonReentrant
_safeTransfer	Internal	Can Modify State	-
_safeTransferFrom	Internal	Can Modify State	-

TimelockedTransparentUpgradableProxy			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Payable	ERC1967Proxy
_proxyAdmin	Internal	-	-
_fallback	Internal	Can Modify State	-
_dispatchUpgradeToAndCall	Private	Can Modify State	-

Token			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	ERC20

VeFunderGauge			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
rewardsList	External	-	-
rewardsListLength	External	-	-
lastTimeRewardApplicable	Public	-	-
rewardData	External	-	-

VeFunderGauge			
getReward	Public	Can Modify State	nonReentrant
left	Public	-	-
whitelistReward	External	Can Modify State	-
removeRewardWhitelist	External	Can Modify State	-
notifyRewardAmount	External	Can Modify State	nonReentrant
isWhitelisted	Public	-	-
_safeTransfer	Internal	Can Modify State	-
_safeTransferFrom	Internal	Can Modify State	-

VoteModule			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyAccessHub
depositAll	External	Can Modify State	-
deposit	Public	Can Modify State	updateReward nonReentrant
withdrawAll	External	Can Modify State	-
withdraw	Public	Can Modify State	updateReward nonReentrant
notifyRewardAmount	External	Can Modify State	updateReward nonReentrant
getReward	External	Can Modify State	updateReward nonReentrant
_claim	Internal	Can Modify State	-
setCooldownExemption	External	Can Modify State	onlyAccessHub
setNewDuration	External	Can Modify State	onlyAccessHub

VoteModule			
setNewCooldown	External	Can Modify State	onlyAccessHub
delegate	External	Can Modify State	-
setAdmin	External	Can Modify State	-
lastTimeRewardApplicable	Public	-	-
earned	Public	-	-
rewardPerToken	Public	-	-
left	Public	-	-
isDelegateFor	External	-	-
isAdminFor	External	-	-

Voter			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
initializeV2	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	-
setGlobalRatio	External	Can Modify State	onlyGovernance
reset	External	Can Modify State	-
_reset	Internal	Can Modify State	-
poke	External	Can Modify State	-
vote	External	Can Modify State	-
_vote	Internal	Can Modify State	-
_distribute	Internal	Can Modify State	-

Voter			
getVotes	External	-	-
setGovernor	External	Can Modify State	onlyGovernance
whitelist	Public	Can Modify State	onlyGovernance
revokeWhitelist	Public	Can Modify State	onlyGovernance
killGauge	Public	Can Modify State	onlyGovernance
reviveGauge	Public	Can Modify State	onlyGovernance
stuckEmissionsRecovery	External	Can Modify State	onlyGovernance
whitelistGaugeRewards	External	Can Modify State	onlyGovernance
removeGaugeRewardWhitelist	External	Can Modify State	onlyGovernance
removeFeeDistributorReward	External	Can Modify State	onlyGovernance
getPeriod	Public	-	-
createSigmaGauge	External	Can Modify State	onlyGovernance
createVeFunderGauge	External	Can Modify State	onlyGovernance
setGaugePreallocation	External	Can Modify State	onlyGovernance
claimIncentives	External	Can Modify State	-
claimRewards	External	Can Modify State	-
claimLegacyRewardsAndExit	External	Can Modify State	-
notifyRewardAmount	External	Can Modify State	-
distribute	Public	Can Modify State	nonReentrant
distributeForPeriod	Public	Can Modify State	nonReentrant
distributeAll	External	Can Modify State	-
batchDistributeByIndex	External	Can Modify State	-

Voter			
getAllGauges	External	-	-
getAllFeeDistributors	External	-	-
isGauge	External	-	-
isFeeDistributor	External	-	-
_claimablePerPeriod	Internal	-	-
_getTotalPreallocationForPeriod	Internal	-	-
_getCurrentTotalGaugePreallocation	Internal	-	-

SigmaAirdrop			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
claim	External	Can Modify State	nonReentrant
batchClaim	External	Can Modify State	nonReentrant
_processClaim	Internal	Can Modify State	-
emergencyWithdraw	External	Can Modify State	onlyGovernance
isSaltUsed	External	-	-
getMessageHash	External	-	-
getEthSignedMessageHash	External	-	-

x33			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer

x33			
_authorizeUpgrade	Internal	Can Modify State	onlyAccessHub
submitVotes	External	Can Modify State	onlyOperator
compound	External	Can Modify State	onlyOperator
claimRebase	External	Can Modify State	onlyOperator
claimIncentives	External	Can Modify State	onlyOperator
swapIncentiveViaAggregator	External	Can Modify State	nonReentrant onlyOperator
unlock	External	Can Modify State	onlyOperator
rescue	External	Can Modify State	nonReentrant onlyAccessHub
transferOperator	External	Can Modify State	onlyAccessHub
whitelistAggregator	External	Can Modify State	onlyAccessHub
totalAssets	Public	-	-
ratio	Public	-	-
getPeriod	Public	-	-
isUnlocked	Public	-	-
isCooldownActive	Public	-	-
_deposit	Internal	Can Modify State	whileNotLocked
_withdraw	Internal	Can Modify State	-

XShadow			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	onlyGovernance

XShadow			
startRebase	External	Can Modify State	onlyGovernance
pause	External	Can Modify State	onlyGovernance
unpause	External	Can Modify State	onlyGovernance
_update	Internal	Can Modify State	-
_isExempted	Internal	-	-
convertEmissionsToken	External	Can Modify State	whenNotPaused
rebase	External	Can Modify State	whenNotPaused
exit	External	Can Modify State	whenNotPaused
createVest	External	Can Modify State	whenNotPaused
exitVest	External	Can Modify State	whenNotPaused
operatorRedeem	External	Can Modify State	onlyGovernance
rescueTrappedTokens	External	Can Modify State	onlyGovernance
migrateOperator	External	Can Modify State	onlyGovernance
setExemption	External	Can Modify State	onlyGovernance
setExemptionTo	External	Can Modify State	onlyGovernance
getBalanceResiding	Public	-	-
usersTotalVests	Public	-	-
getVestInfo	Public	-	-
isExempt	External	-	-
shadow	External	-	-

SigmaFeeDistributorFactory			
Function Name	Visibility	Mutability	Modifiers

SigmaFeeDistributorFactory			
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	-
createFeeDistributor	External	Can Modify State	-

SigmaGaugeFactory			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	-
createGauge	External	Can Modify State	-

VeFunderGaugeFactory			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
_authorizeUpgrade	Internal	Can Modify State	-
createGauge	External	Can Modify State	-

PoolManager			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	initializer
getPoolInfo	External	-	-

PoolManager			
operate	External	Can Modify State	-
operate	Public	Can Modify State	onlyRegisteredPool nonReentrant whenNotPaused onlyTopLevelCall lock
redeem	External	Can Modify State	onlyRegisteredPool nonReentrant whenNotPaused lock
rebalance	External	Can Modify State	onlyRegisteredPool nonReentrant whenNotPaused onlyRole
rebalance	External	Can Modify State	onlyRegisteredPool nonReentrant whenNotPaused onlyRole lock
liquidate	External	Can Modify State	onlyRegisteredPool nonReentrant whenNotPaused onlyRole lock
harvest	External	Can Modify State	onlyRegisteredPool onlyRole nonReentrant
borrow	External	Can Modify State	onlyCounterparty onlyRegisteredPool nonReentrant
repay	External	Can Modify State	onlyCounterparty onlyRegisteredPool nonReentrant
repayByCreditNote	External	Can Modify State	onlyCounterparty onlyRegisteredPool nonReentrant
liquidateShortPool	External	Can Modify State	onlyCounterparty onlyRegisteredPool nonReentrant
reduceDebt	External	Can Modify State	onlyRegisteredPool onlyRole nonReentrant
setPause	External	Can Modify State	onlyRole
registerPool	External	Can Modify State	onlyRole
updateRateProvider	External	Can Modify State	onlyRole
updatePoolCapacity	External	Can Modify State	onlyRole onlyRegisteredPool
updateThreshold	External	Can Modify State	onlyRole
updateShortBorrowCapacityRatio	External	Can Modify State	onlyRole

PoolManager			
_takeAccumulatedPoolFee	Internal	Can Modify State	-
_updatePoolCapacity	Internal	Can Modify State	-
_updateThreshold	Internal	Can Modify State	-
_scaleUp	Internal	-	-
_scaleUp	Internal	-	-
_scaleDown	Internal	-	-
_scaleDownRoundingUp	Internal	-	-
_scaleDown	Internal	-	-
_handleSupply	Internal	Can Modify State	-
_handleWithdraw	Internal	Can Modify State	-
_handleBorrow	Internal	Can Modify State	-
_handleRepay	Internal	Can Modify State	-
_beforeRebalanceOrLiquidate	Internal	-	-
_afterRebalanceOrLiquidate	Internal	Can Modify State	-
_changePoolCollateral	Internal	Can Modify State	-
_changePoolDebts	Internal	Can Modify State	-
_getTokenScalingFactor	Internal	-	-
_getPoolCollateralInfo	Internal	-	-
_transferCollateralOut	Internal	Can Modify State	-

PoolManager			
_transferFrom	Internal	Can Modify State	-

StrategyBaseUpgradeable			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
__StrategyBaseUpgradeable_init	Internal	Can Modify State	onlyInitializing
__StrategyBaseUpgradeable_init_unchained	Internal	Can Modify State	onlyInitializing
harvest	External	Can Modify State	onlyRole
execute	External	Can Modify State	onlyOrigin
_harvest	Internal	Can Modify State	-

YieldStrategy			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	StrategyBaseUpgradeable
initialize	External	Can Modify State	initializer
totalSupply	Public	-	-
deposit	External	Can Modify State	onlyOrigin
withdraw	External	Can Modify State	onlyOrigin
kill	External	Can Modify State	onlyOrigin
_harvest	Internal	Can Modify State	-
reducePrincipal	External	Can Modify State	onlyRole
rescueTokens	External	Can Modify State	onlyRole
updateTreasury	External	Can Modify State	onlyRole

ShortPositionOperateFacet			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	MorphoFlashLoanFacetBase
openOrAddShortPosition	External	Payable	nonReentrant onlyTopLevelCall
closeOrRemoveShortPosition	External	Can Modify State	nonReentrant onlyTopLevelCall
_swap	Internal	Can Modify State	-
_checkPositionDebtRatio	Internal	-	-

4.3 Vulnerability Summary

[N1] [Medium] Risks of excessive privilege

Category: Authority Control Vulnerability Audit

Content

1.In the PoolManager contract, the `DEFAULT_ADMIN_ROLE` role has the right to grant and revoke all roles; the `OPERATOR_ROLE` role is responsible for configuring all runtime parameters; the `EMERGENCY_ROLE` role can only control the pause and unpause of the contract; the `HARVESTER_ROLE` role is responsible for triggering profit distribution; the `SP_ROLE` role is responsible for soft/hard liquidation operations; and the `DEBT_REDUCER_ROLE` role can destroy the fxUSD held by the caller and simultaneously reduce the on-chain debt record of the specified pool.

- contracts/core/PoolManager.sol#L745–L754, L759–L764, L770–L776, L780–L782, L787–L792, L736–L739, L559–L640, L446–L475, L478–L507, L510–L556, L718–L728

```
registerPool(address pool, uint96 collateralCapacity, uint96 debtCapacity)

updateRateProvider(address token, address provider)

updatePoolCapacity(address pool, uint96 collateralCapacity, uint96 debtCapacity)

updateThreshold(uint256 newThreshold)

updateShortBorrowCapacityRatio(address longPool, uint256 newRatio)

setPause(bool status)
```

```
    harvest(address pool)

    rebalance(address pool, address receiver, int16 tick, uint256 maxFxUSD, uint256
maxStable)

    rebalance(address pool, address receiver, uint256 maxFxUSD, uint256 maxStable)

    liquidate(address pool, address receiver, uint256 maxFxUSD, uint256 maxStable)

    reduceDebt(address pool, uint256 amount)
```

2.In the YieldStrategy contract, the `Origin` role controls the inflow and outflow of funds and the lifecycle of the strategy; `DEFAULT_ADMIN_ROLE` is responsible for maintaining the parameters of the strategy contract and handling emergencies.

- contracts/fund/strategy/YieldStrategy.sol#L43–L51, L53–L62, L64–L71, L40–L46, L82–L93, L95–L100, L102–L109

```
    deposit(uint256 amount)

    withdraw(uint256 amount, address recipient)

    kill()

    execute(address to, uint256 value, bytes calldata data)

    reducePrincipal(uint256 amount)

    rescueTokens(address token, uint256 amount)

    updateTreasury(address _treasury)
```

3.In the AccessHub contract, the `owner` role can transfer ownership, upgrade the contract, arbitrarily call target contracts, set operators, and perform key governance operations such as emergency fund withdrawal and VoteModule parameter setting. During initialization, the owner role simultaneously acquires the `DEFAULT_ADMIN_ROLE` and `PROTOCOL_OPERATOR` roles. The `DEFAULT_ADMIN_ROLE` role can forcibly remove inactive users; the `PROTOCOL_OPERATOR` role is responsible for the daily governance operations of the protocol.

- dao/contracts/AccessHub.sol#L81–L91, L94–L118, L121–L123, L128–L133, L136–L141, L144–L151, L154–L166, L169–L176, L179–L186, L189–L193, L196–L201, L204–L209, L214–L220, L223–L227, L230–L234, L237–L241, L244–L249, L252–L258, L263–L271, L274–L276, L279–L283, L286–L290, L293–L299, L304–L322, L325–L336, L341–L348, L351–L355, L358–L362, L365–L382, L387–L393, L396–L399, L402–L409, L412–L418

```
    reinit(InitParams calldata params)

    initializeVoter(address _shadow, address

    _veFunderGaugeFactory, address _sigmaGaugeFactory, address

    _sigmaFeeDistributorFactory, address _gaugeEmission, address _msig, address
_xShadow, address _voteModule, uint256 _maxTotalSigmaGaugePreallocation, uint256
_maxVeFunderGaugeCap)

    startRebase(address _voteModule, address _voter)

    setNewGovernorInVoter(address _newGovernor)

    createSigmaGauge(address _pool, uint256 _preallocationBps)

    createVeFunderGauge(address _receiver, uint256 _maxEmission, address _pool,
uint256 _preallocationBps)
governanceWhitelist(address[] calldata _token, bool[] calldata _whitelisted)

    killGauge(address[] calldata _pairs)

    reviveGauge(address[] calldata _pairs)

    setEmissionsRatioInVoter(uint256 _pct)

    retrieveStuckEmissionsToGovernance(address _gauge, uint256 _period)

    setGaugePreallocation(address _gauge, uint256 _preallocationBps)

    transferWhitelistInXShadow(address[] calldata _who, bool[] calldata _whitelisted)

    toggleXShadowGovernance(bool enable)

    operatorRedeemXShadow(uint256 _amount)

    migrateOperator(address _operator)

    rescueTrappedTokens(address[] calldata _tokens, uint256[] calldata _amounts)
```

```
    setExemptionToInXShadow(address[] calldata _who, bool[] calldata _whitelisted)

    kickoffGaugeEmission(address _shadow, uint256 _initialWeeklyEmissions, uint256
_initialMultiplier, uint256 _multiplierUpdatePeriod, address _xShadow)

    startGaugeEmissions(address _voter)

    setEmissionsMultiplier(uint256 _multiplier)

    setGaugeActive(bool _isGaugeActive)

    emergencyWithdrawFromGaugeEmission(address _token, address _to, uint256 _amount)

    augmentGaugeRewardsForPair(address[] calldata _pools, address[] calldata _rewards,
bool[] calldata _addReward)
removeFeeDistributorRewards(address[] calldata _pools, address[] calldata _rewards)

    setCooldownExemption(address[] calldata _candidates, bool[] calldata _exempt)

    setNewRebaseStreamingDuration(uint256 _newDuration)

    setNewVoteModuleCooldown(uint256 _newCooldown)

    kickInactive(address[] calldata _nonparticipants)

    execute(address _target, bytes calldata _payload)

    setNewOwner(address _owner)

    setNewOperator(address _operator)

    upgradeContract(address _proxy, address _newImplementation, bytes calldata _data)
```

Solution

In the short term, transferring the ownership of core roles to time-locked contracts and managing them by multi-signatures is an effective solution to avoid single-point risks. However, in the long run, a more reasonable solution is to implement a permission separation strategy and set up multiple privileged roles to manage each privileged function separately. Permissions involving user funds and contract core parameter updates should be managed by the community, while permissions involving emergency contract suspensions can be managed by EOA addresses. This ensures the safety of user funds while responding quickly to threats.

Status

Acknowledged

[N2] [Suggestion] Redundant code

Category: Others

Content

In the ShortPositionOperateFacet contract, the `_swap` function is not used.

- contracts/periphery/facets/ShortPositionOperateFacet.sol#L153-L186

```
    _swap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minOut,
    address swapTarget,
    bytes memory swapData
  )
```

Solution

It is recommended to remove redundant code.

Status

Fixed

[N3] [High] Owner transfer not synchronized AccessControl role

Category: Authority Control Vulnerability Audit

Content

In the AccessHub contract, `setNewOwner()` did not simultaneously transfer the `DEFAULT_ADMIN_ROLE` and `PROTOCOL_OPERATOR` roles when transferring the `owner` role, resulting in the old `owner` address still having the permissions of these two roles.

- dao/contracts/AccessHub.sol#L396-L399

```
    function setNewOwner(address _owner) external onlyOwner {
        require(owner != _owner, SAME_ADDRESS());
```

```
        owner = _owner;
    }
```

Solution

It is recommended to transfer the DEFAULT_ADMIN_ROLE and PROTOCOL_OPERATOR role permissions of the old owner address when transferring the owner role.

Status

Fixed

[N4] [Suggestion] Missing zero address check

Category: Others

Content

In the AccessHub contract, `initialize()`, `reinit()`, `setNewOwner()`, `setNewOperator()` lack zero address checks for address type parameters.

- dao/contracts/AccessHub.sol#L58-L78, L81-L91, L396-L399, L402-L409

```
    initialize(InitParams calldata params)

    reinit(InitParams calldata params)

    setNewOwner(address _owner)

    setNewOperator(address _operator)
```

Solution

It is recommended to add a zero address check.

Status

Fixed

[N5] [Suggestion] Missing event log

Category: Others

Content

In the AccessHub contract, `reinit()`, `setNewOwner()`, and `setNewOperator()` modify important parameters of the contract, but event logging is missing.

- dao/contracts/AccessHub.sol#L91-L91, L396-L399, L402-L409

```
reinit(InitParams calldata params)

setNewOwner(address _owner)

setNewOperator(address _operator)
```

Solution

It is recommended to add event logging.

Status

Fixed

[N6] [Suggestion] Ignore return value

Category: Others

Content

In x33 contracts, `initialize()` ignores the return values of `shadow.approve()` and `IERC20(_xShadow).approve()`.

- dao/contracts/xShadow/XShadow.sol#L72-L101

```
function initialize(
    address _operator,
    address _accessHub,
    address _xShadow,
    address _voter,
    address _voteModule
) external initializer {
    //...
    shadow.approve(address(xShadow), type(uint256).max);
    IERC20(_xShadow).approve(address(voteModule), type(uint256).max);
}
```

Solution

It is recommended to check the function's return value.

Status

Fixed

[N7] [Suggestion] Missing array length consistency check

Category: Others

Content

In the AccessHub contract, `setCooldownExemption()` lacks a length consistency check for `_candidates` and `_exempt`.

- dao/contracts/AccessHub.sol#L341-L348

```
function setCooldownExemption(
    address[] calldata _candidates,
    bool[] calldata _exempt
) external onlyOwner {
    for (uint256 i; i < _candidates.length; ++i) {
        voteModule.setCooldownExemption(_candidates[i], _exempt[i]);
    }
}
```

Solution

It is recommended to add an array length consistency check.

Status

Fixed

[N8] [High] The risk of conflict when both initializeV2() and initialize() use the initializer modifier

Category: Design Logic Audit

Content

In the Voter contract, the `initializer` modifier was misused in `initializeV2()`, causing a revert to occur every time the contract is upgraded. This is because `_initialized` was set to 1 in the proxy contract's storage during V1 deployment, and the `initializer`'s pre-check requires this value to be 0 to pass. After the upgrade, this condition is no longer met, making it impossible for newly added critical state variables such as `accessHub` to be set.

- dao/contracts/Voter.sol#L169-L207

```
function initializeV2(
    address _accessHub,
    address _shadow,
    address _veFunderGaugeFactory,
    address _sigmaGaugeFactory,
    address _sigmaFeeDistributorFactory,
    address _gaugeEmission,
    address _msig,
    address _xShadow,
    address _voteModule,
    uint256 _maxTotalSigmaGaugePreallocation,
    uint256 _maxVeFunderGaugeCap
) external initializer {
    //...
}
```

Solution

It is recommended to use the reinitializer(2) modifier for initializeV2, or ensure that _authorizeUpgrade is compatible with both initialization paths.

Status

Fixed

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002603020001	SlowMist Security Team	2026.02.27 - 2026.03.02	Medium Risk

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 2 high risk, 1 medium risk, 5 suggestion vulnerabilities.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

Official Website

www.slowmist.com

E-mail

team@slowmist.com

Twitter

@SlowMist_Team

Github

https://github.com/slowmist